Retrieval Augmented Generation Assignment Building a Simple Law Chatbot

Tho Quan

Version 1.0, November 9, 2025

1 Problem Statement

The goal of this assignment is to build a law question-answering system that leverages retrieval-augmented generation (RAG) techniques to improve over a provided baseline. Students are expected to focus on enhancing the retrieval component and the integration of retrieved information into the answer generation process.

The main problem is that naive baselines often provide limited accuracy because:

- Relevant information may not be retrieved efficiently from the document corpus.
- Retrieved context may not be effectively used to answer questions.

Students are tasked with exploring and applying various RAG techniques, including:

- Choice and tuning of embeddings for better semantic search.
- Design and optimization of retrieval methods (e.g., vector-based retrieval, hybrid retrieval).
- Methods to select, rank, or filter retrieved passages to improve answer relevance.

The focus is on improving measurable metrics compared to the baseline, demonstrating the effectiveness of RAG techniques in a controlled assignment setting.

2 RAG Overview and Evaluation Metrics

Retrieval-Augmented Generation [1] is a framework that combines retrieval of relevant documents with a generation process to produce more accurate and grounded answers. In this assignment, students are expected to work primarily on the retrieval component and the integration of retrieved passages into the answer.

Key aspects of RAG in this assignment:

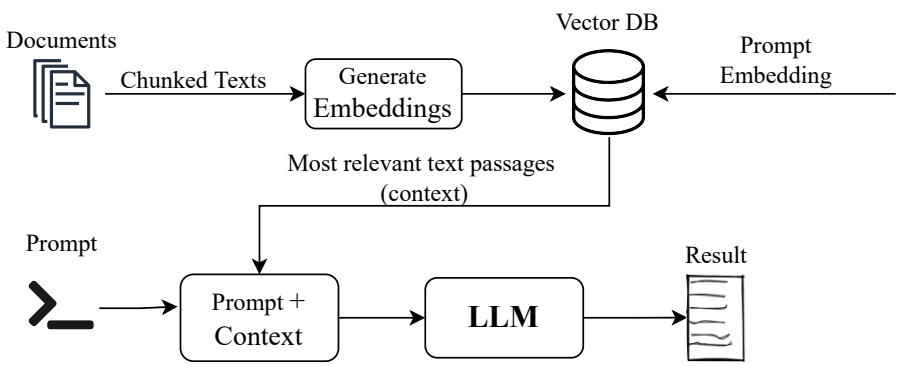

Figure 1: Overview of a Retrieval-Augmented Generation system. The system retrieves relevant documents from a knowledge base and integrates them into the answer generation process to produce more accurate and context-aware responses.

- Document Retrieval: Efficiently finding passages that are most relevant to a given question from a legal corpus.
- Context Integration: Structuring and selecting retrieved passages so that they contribute effectively to answer generation.
- Evaluation: Measuring how well the RAG system improves over the baseline using objective metrics.

Evaluation Metrics:

We evaluate the quality of generated answers using a set of similarity metrics that compare the prediction $\hat{y}$ with the reference answer y.

- Cosine Similarity: Measures the similarity between vector embeddings of the generated answer and the ground truth [3]. Given embedding vectors $\mathbf{v}_{\hat{y}}$ and $\mathbf{v}_y$, the cosine similarity is defined as:
$$\text{CosineSim}(\hat{y}, y) = \frac{\mathbf{v}_{\hat{y}} \cdot \mathbf{v}_y}{\|\mathbf{v}_{\hat{y}}\| \, \|\mathbf{v}_y\|}$$
- Token Overlap / Jaccard Similarity: Measures the word-level overlap between the generated answer and the reference answer[1]. Let $W_{\hat{y}}$ and W_y be the sets of tokens in $\hat{y}$ and y, then the Jaccard similarity is:
$$\text{Jaccard}(\hat{y}, y) = \frac{|W_{\hat{y}} \cap W_y|}{|W_{\hat{y}} \cup W_y|}$$
Token Overlap score can also be defined as the fraction of reference tokens present in the generated answer:
$$\text{TokenOverlap}(\hat{y}, y) = \frac{|W_{\hat{y}} \cap W_y|}{|W_y|}$$
- BLEU: Measures n-gram precision of the generated answer with respect to the reference answer [4]. For a simple unigram BLEU:
$$\text{BLEU}(\hat{y}, y) = BP \cdot \frac{|W_{\hat{y}} \cap W_y|}{|W_{\hat{y}}|}$$
where BP is the brevity penalty:
$$BP = \begin{cases} 1, & \text{if } |\hat{y}| \geq |y| \\ \exp(1 - |y|/|\hat{y}|), & \text{if } |\hat{y}| < |y| \end{cases}$$
- ROUGE-L: Measures the longest common subsequence (LCS) between $\hat{y}$ and y, capturing sequence-level similarity [2]. Let $LCS(\hat{y}, y)$ be the length of the longest common subsequence:
$$P = \frac{LCS(\hat{y}, y)}{|\hat{y}|}, \quad R = \frac{LCS(\hat{y}, y)}{|y|}$$
$$\text{ROUGE-L} = \frac{2 \cdot P \cdot R}{P + R}$$

Students are expected to use these metrics to compare their RAG system against the baseline and provide evidence of improvement.

3 Baseline

For this assignment, we provide a simple baseline RAG system together with a collection of legal domain documents and a RES dataset containing question-answer pairs. The baseline system is designed to serve as a starting point for students to explore and improve upon.

In the baseline, each document is first split into chunks of approximately 2000 characters with an overlap of 50 characters between consecutive chunks. These chunks are then converted into

[1] https://deepai.org/machine-learning-glossary-and-terms/jaccard-index

embeddings using the Vietnamese SBERT[2] model and stored in a local FAISS[3] vector database. At query time, the system retrieves the top $k = 5$ most relevant chunks from the database based on cosine similarity between the question embedding and the document embeddings. The retrieved passages are subsequently provided as context to the LLM, specifically Qwen/Qwen1.5-1.8B[4], which generates the final answer.

For the assignment, students are free to select any embedding model they consider appropriate, as well as any LLM with fewer than 4 billion parameters that fits their available resources. The main objective is to leverage retrieval-augmented generation techniques to improve upon the baseline results according to the grading criteria.

3.1 Documents

The assignment provides two main datasets for the RAG system:

- **Legal Documents:** A collection of about 16,440 text files covering various legal domains (e.g., civil, labor, administrative) from multiple levels of authorities. Students can explore the corpus to understand its structure, document distribution, and common terminology, which can guide retrieval strategies and embedding choices.
- **RES Dataset (Questions and Answers):** Approximately 2,745 question-answer pairs for evaluating the RAG system. Students may use a smaller subset if needed, but **at least 100 pairs must be tested**, and the number used should be reported in the submission.

The baseline code provides a simple way to load and adjust the number of Q&A pairs for testing:

Loading RES Dataset

```python
import pandas as pd

# Load RES dataset
df = pd.read_csv("res.csv")

# Adjust the number of Q&A pairs to evaluate (minimum 100)
df = df.head(100)
print(f"Total number of questions: {len(df)}")
```

This setup allows students to systematically test their retrieval-augmented generation system. By analyzing performance on these Q&A pairs, students can identify strengths and weaknesses in their retrieval methods, embedding selection, and how well the retrieved passages contribute to accurate answer generation.

Source: Google Drive folder for documents

3.2 Baseline Source Code

As introduced above, the provided baseline is designed to be simple, clear, and easy to understand. It is built using the LangChain framework, with FAISS as the vector database, Qwen/Qwen1.5-1.8B as the LLM, and Vietnamese-SBERT as the embedding model. The baseline workflow is straightforward: documents are embedded and stored in FAISS, and queries are answered by retrieving the top relevant passages and generating an answer via the LLM.

Some key code snippets from the baseline are provided below for reference:

[2] https://huggingface.co/keepitreal/vietnamese-sbert
[3] https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/
[4] https://huggingface.co/Qwen/Qwen1.5-1.8B

Embedding Model Initialization

```python
from langchain_community.embeddings import HuggingFaceEmbeddings

EMBEDDING_MODEL="keepitreal/vietnamese-sbert"  # Choose your embedding model
print(f"Loading embedding model: {EMBEDDING_MODEL}")

embeddings = HuggingFaceEmbeddings(
    model_name=EMBEDDING_MODEL,
    model_kwargs={"device": "cuda"}
)
```

LLM Initialization

```python
from langchain_huggingface import HuggingFacePipeline

llm = HuggingFacePipeline.from_model_id(
    model_id="Qwen/Qwen1.5-1.8B",  # Choose your model
    task="text-generation",
    device=0,  # GPU 0
    model_kwargs={"temperature": 0, "max_length": 1024}
)
```

Students are free to choose any embedding model or LLM under 4 billion parameters that best fits their computational resources and assignment constraints. The main objective is to focus on improving the RAG system's retrieval and integration pipeline, while achieving better results than the provided baseline according to the grading metrics.

Source code: Colab source code of the Baseline

3.3 Running Baseline Instructions

To run the baseline system, students need access to the document link and the source code link provided in Sections 3.1 and 3.2. The baseline can be executed on Google Colab by following these two steps:

1. **Copy the source code:** Open the Colab notebook provided in Section 3.2 and create a copy in your own Google Drive. This allows you to modify and run the notebook independently.
2. **Add the document folder:** Access the Google Drive link provided in Section 3.1 and add the original `RAG Assignment` folder as a shortcut in your Google Drive. This ensures that the Colab notebook you copied in Step 1 can access all the necessary documents via this shortcut and run correctly.

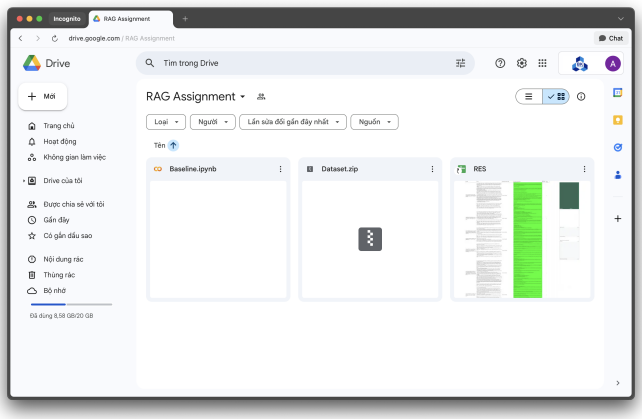

Figure 2: Step 2.1 Access the RAG Assignment folder.

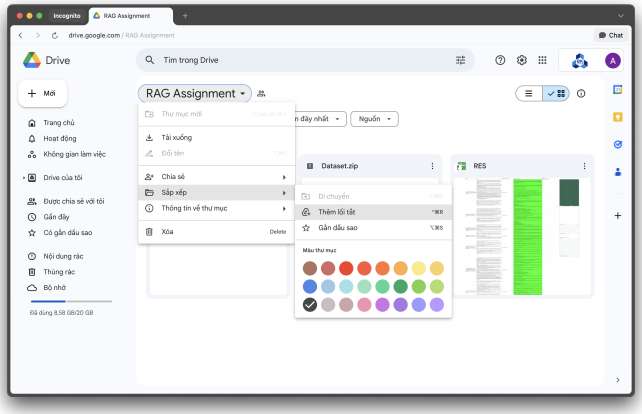

Figure 3: Step 2.2 Choose Arrange & Add Shortcut.

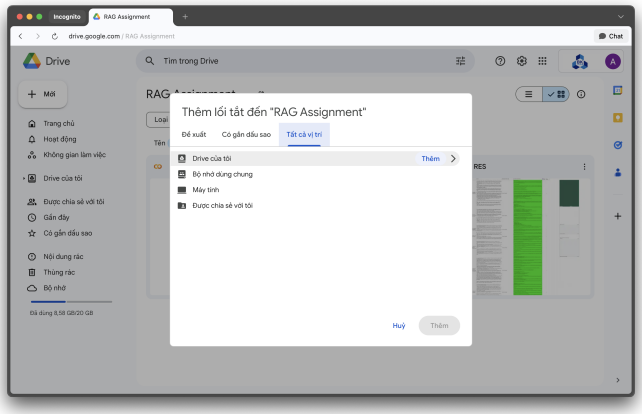

Figure 4: Step 2.3 Add Shortcut to myDrive.

Following these steps will prepare your environment for running the baseline RAG system, including loading the legal documents, RES dataset, embeddings, and the LLM.

4 Grading

- **Improvements compared to baseline – 3 pts:**
  This criterion evaluates how much your RAG system improves over the provided baseline in terms of measurable metrics. Students are expected to demonstrate objective improvements on at least one of the following:
  - Cosine similarity between generated answers and ground truth embeddings.
  - Token-level overlap (Token Overlap, Jaccard Similarity).
  - BLEU or ROUGE-L scores.
- **Demonstration – 2 pts:**
  Students must demonstrate their RAG system on a small set of example questions. Points are awarded based on:
  - Clarity and readability of the responses.
  - Relevance and correctness of the answers.
  - Ease of running the demo (scripts/notebooks should run without major errors).

- **Report – 5 pts:**
  The written report should clearly explain your approach, experimental results, and analysis. Points are divided as follows:
  - **Data Exploration & Understanding – 1 pt:**
    Demonstrate that you have explored the dataset, analyzed its characteristics (e.g., number of questions, diversity, difficulty), and understood its structure.
  - **Method to improve RAG vs baseline – 3 pts:**
    Describe the techniques you applied to improve retrieval or generation, including but not limited to:
    * Choice or tuning of embeddings and retrievers.
    * Prompt engineering for the LLM.
    * Integration of retrieved context with the generation step.
    * Any other improvements supported by quantitative metrics.

    Students must show evaluation results indicating the improvements over baseline metrics.
  - **Format & Clarity – 1 pt:**
    The report should be well-structured, clearly written, with proper headings, figures/tables if needed, and reproducible instructions for running your RAG system.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[2] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics.

[3] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval.* Cambridge University Press, 2008.

[4] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In Pierre Isabelle, Eugene Charniak, and Dekang Lin, editors, *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA, July 2002. Association for Computational Linguistics.